

Video to 3D Model — Ubuntu Setup Guide

Complete pipeline for converting a video into a 3D model using **COLMAP** (Structure-from-Motion) and **Instant-NGP** (NeRF).

Tested on: Ubuntu 22.04, GTX 1660 Ti 6GB, CUDA 12.6, Driver 560.35

Prerequisites

Tool	Purpose
NVIDIA GPU + Driver	CUDA acceleration
CUDA Toolkit 12.x	NVCC compiler for Instant-NGP
CMake 3.24+	Build system
COLMAP	Structure-from-Motion + camera pose estimation
Instant-NGP	NeRF training + mesh export
MeshLab	View and inspect 3D meshes
Blender (optional)	Clean up mesh + measure volume

Step 1 — Install System Packages

```
sudo apt update

sudo apt install -y build-essential git python3-dev python3-pip \
    libboost-all-dev libeigen3-dev libflann-dev libfreeimage-dev \
    libgoogle-glog-dev libgtest-dev libsqlite3-dev libglew-dev \
    qtbase5-dev libqt5opengl5-dev libcgald-dev libceres-dev \
    ffmpeg colmap meshlab imagemagick
```

Step 2 — Install CUDA Toolkit (if not already installed)

Check if nvcc exists:

```
nvcc --version
```

If missing or version is below 12.x:

```
# Remove old toolkit
sudo apt remove -y nvidia-cuda-toolkit
sudo apt autoremove -y

# Install CUDA 12.6 (adjust for your driver version)
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/c
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update
sudo apt install -y cuda-toolkit-12-6

# Add to PATH
echo 'export PATH=/usr/local/cuda-12.6/bin:$PATH' >> ~/.bashrc
echo 'export LD_LIBRARY_PATH=/usr/local/cuda-12.6/lib64:$LD_LIBRARY_PATH' >> ~/.b
source ~/.bashrc

# Verify
nvcc --version
```

Step 3 — Upgrade CMake

The default Ubuntu CMake (3.22) is too old. Upgrade via pip:

```
pip install cmake --upgrade
hash -r
cmake --version    # should be 3.24+
```

Step 4 — Create Python Virtual Environment

```
python3 -m venv ~/v2model
source ~/v2model/bin/activate
pip install --upgrade pip
```

```
pip install opencv-python numpy matplotlib Pillow
```

Step 5 — Clone Repositories

```
# Clone the Video-To-3D-Model project
git clone https://github.com/APEXPRE123207/Video-To-3D-Model.git

# Clone Instant-NGP (use a path WITHOUT spaces)
git clone --recursive https://github.com/NVlabs/instant-ngp
```

IMPORTANT: Do NOT clone into a path with spaces (e.g. `New Volume`). If your drive has spaces, create a symlink:

```
ln -s "/media/user/New Volume/instant-ngp" ~/instant-ngp
```

Step 6 — Build Instant-NGP

```
cd instant-ngp

# Create a tmp directory on a drive with free space if root (/) is full
mkdir -p /path/to/large/drive/tmp

# Build (GTX 1660 Ti = compute capability 75)
TMPDIR="/path/to/large/drive/tmp" cmake . -B build \
    -DCMAKE_BUILD_TYPE=RelWithDebInfo \
    -DCMAKE_CUDA_ARCHITECTURES=75

TMPDIR="/path/to/large/drive/tmp" cmake --build build \
    --config RelWithDebInfo -j$(nproc)
```

Common CUDA architectures:

- GTX 1660 Ti / RTX 2060–2080: 75
- RTX 3060–3090: 86
- RTX 4060–4090: 89

Verify the build:

```
ls build/instant-ngp # executable should exist
```

Step 7 — Copy and Fix Project Scripts

```
cd instant-ngp
cp /path/to/Video-To-3D-Model/frames.py .
cp /path/to/Video-To-3D-Model/visualize_COLMAP.py .
cp /path/to/Video-To-3D-Model/blender_volume.py .
```

Create fixed `video_to_COLMAP.py` (Linux-compatible)

The original script uses Windows paths. Replace it with:

```
# video_to_COLMAP.py
import subprocess
from frames import extract_frames
from visualize_COLMAP import plot_camera_positions_with_direction
import shutil
import os

if __name__ == "__main__":
    video_path = input("Insert path to the video file, including the file itself.

PROJECT_ROOT = os.path.dirname(os.path.abspath(__file__))
frames_save_path = os.path.join(PROJECT_ROOT, "data", "v2obj", "images")

if not os.path.exists(frames_save_path):
    os.makedirs(frames_save_path)

number_of_samples = extract_frames(video_path, frames_save_path)
print("{} samples created".format(number_of_samples))

images_path = frames_save_path
colmap2nerf_script = os.path.join(PROJECT_ROOT, "scripts", "colmap2nerf.py")

colmap2nerf_arguments = [
    '--colmap_matcher', 'exhaustive',
    '--run_colmap',
    '--aabb_scale', '32',
    '--images', images_path
]
```

```

command = ['python', colmap2nerf_script] + colmap2nerf_arguments
subprocess.run(command)

_flag = input("Do you want to visualize the camera position from COLMAP? (y/n) ")
_flag = True if _flag == 'y' or _flag == 'Y' else False

if _flag:
    with open('transforms.json', 'r') as file:
        json_data = file.read()
        plot_camera_positions_with_direction(json_data)

transforms_file = 'transforms.json'
destination_dir = os.path.join(PROJECT_ROOT, "data", "v2obj")

if os.path.exists(transforms_file):
    if not os.path.exists(destination_dir):
        os.makedirs(destination_dir)
    shutil.move(transforms_file, destination_dir)

```

Create fixed `correction1.py`

Place in `data/v2obj/correction1.py`. Check the original — if it has Windows backslashes (`\\`), replace them with forward slashes (`/`).

Create fixed `correction2.py`

```

# correction2.py
from PIL import Image
import os

# Update this path to match your setup
image_dir = "./images"

for fname in os.listdir(image_dir):
    fpath = os.path.join(image_dir, fname)
    try:
        with Image.open(fpath) as img:
            img.verify()
            print(f"✅ OK: {fname}")
    except Exception as e:
        print(f"❌ ERROR: {fname} - {e}")

```

Step 8 — Record Your Video

Guidelines for best results:

- Resolution: **1080p 60fps** or **4K** preferred
 - Start from the **top** of the object, slowly spiral downward
 - Complete a full **360° orbit** covering every angle
 - Move **slowly and smoothly** — no shaking, no jump cuts
 - Ensure **even lighting**, minimize shadows
 - Keep the object **centered** in the frame throughout
-

Step 9 — Extract Frames

```
source ~/v2model/bin/activate
cd /path/to/instant-ngp
python video_to_COLMAP.py
```

When prompted:

1. Enter the full path to your video
2. Enter sample rate: 1 (1 frame per second)

Note: If COLMAP crashes with a Qt error inside the venv, run COLMAP manually outside the venv (see Step 10).

Step 10 — Run COLMAP Manually (if Step 9 COLMAP fails)

This happens because OpenCV's Qt conflicts with COLMAP's Qt. Fix: run COLMAP outside the venv.

```
deactivate
cd /path/to/instant-ngp

# Feature extraction
colmap feature_extractor \
  --ImageReader.camera_model OPENCV \
  --ImageReader.single_camera 1 \
  --SiftExtraction.estimate_affine_shape=true \
```

```
--SiftExtraction.domain_size_pooling=true \  
--database_path colmap.db \  
--image_path data/v2obj/images  
  
# Feature matching  
colmap exhaustive_matcher \  
    --database_path colmap.db  
  
# Sparse reconstruction  
mkdir -p colmap_sparse  
colmap mapper \  
    --database_path colmap.db \  
    --image_path data/v2obj/images \  
    --output_path colmap_sparse  
  
# Export to text format  
mkdir -p colmap_text  
colmap model_converter \  
    --input_path colmap_sparse/0 \  
    --output_path colmap_text \  
    --output_type TXT
```

Step 11 — Generate transforms.json

```
source ~/v2model/bin/activate  
cd /path/to/instant-ngp  
  
python scripts/colmap2nerf.py \  
    --images data/v2obj/images \  
    --colmap_db colmap.db \  
    --text colmap_text \  
    --aabb_scale 32 \  
    --out data/v2obj/transforms.json
```

Step 12 — Validate Images

```
cd /path/to/instant-ngp/data/v2obj  
python correction1.py  
python correction2.py
```

All images should show . Remove any that show .

Step 13 — (Optional) Resize Frames for Low VRAM

If you have a 6GB GPU and many frames (300+) or 4K images:

```
cd /path/to/instant-ngp/data/v2obj/images
for f in *.jpg *.png; do
    convert "$f" -resize 50% "$f"
done
```

Step 14 — Launch Instant-NGP and Train NeRF

```
cd /path/to/instant-ngp
./build/instant-ngp --scene data/v2obj
```

The GUI opens with real-time NeRF training. Wait 1–5 minutes until the scene looks sharp.

Step 15 — Export 3D Mesh

Inside the Instant-NGP GUI:

1. Adjust the **crop box** to tightly surround your object
 2. Set mesh resolution (start with **256**)
 3. Click **Export mesh**
 4. The **.obj** or **.ply** file saves into `data/v2obj/`
-

Step 16 — View in MeshLab

```
meshlab /path/to/instant-ngp/data/v2obj/mesh.obj
```

Step 17 — (Optional) Clean Up in Blender

```
sudo apt install -y blender
blender
```

- 1. File → Import → Wavefront (.obj)
- 2. Delete floating noise artifacts
- 3. Press `N` → adjust dimensions
- 4. Scripting tab → New → paste `blender_volume.py` → Run Script
- 5. Volume is printed in the console

File Reference

File	Purpose
<code>frames.py</code>	Extracts video frames at chosen fps
<code>video_to_COLMAP.py</code>	Orchestrates: frame extraction → COLMAP → <code>transforms.json</code>
<code>visualize_COLMAP.py</code>	Plots camera positions from COLMAP output
<code>correction1.py</code>	Fixes image paths in <code>transforms.json</code>
<code>correction2.py</code>	Validates extracted images aren't corrupted
<code>blender_volume.py</code>	Computes volume of the 3D model in Blender
<code>scripts/colmap2nerf.py</code>	Converts COLMAP output to Instant-NGP <code>transforms.json</code>

Pipeline Summary

```
Video (.mp4)
|
|→ frames.py → Extracted frames (data/v2obj/images/)
|
|→ COLMAP feature_extractor → colmap.db
```

```

├─▶ COLMAP exhaustive_matcher
├─▶ COLMAP mapper → colmap_sparse/
├─▶ COLMAP model_converter → colmap_text/
|
├─▶ colmap2nerf.py → transforms.json
|
├─▶ correction1.py + correction2.py → validated scene
|
├─▶ instant-ngp --scene → NeRF training (real-time)
|
└─▶ Export mesh → .obj / .ply file
    |
    ├─▶ MeshLab (view/inspect)
    └─▶ Blender (clean up + volume)

```

Troubleshooting

Problem	Solution
No CMAKE_CUDA_COMPILER found	Install CUDA toolkit: <code>sudo apt install cuda-toolkit-12-6</code>
CUDA_ARCHITECTURES is empty	Add <code>-DCMAKE_CUDA_ARCHITECTURES=75</code> to cmake command
No space left on device	Set <code>TMPDIR</code> to a drive with free space
CMake symlink fails with spaces in path	Use a path without spaces or create a symlink
COLMAP Qt crash in venv	Run COLMAP commands outside the venv (deactivate first)
<code>correction2.py</code> Windows path error	Replace path with <code>./images</code>
Black/empty NeRF scene	Check <code>transforms.json</code> — image paths must match actual files
CUDA out of memory (6GB GPU)	Resize images to 50%, keep under 200 frames